

Fractal Generative Models

Tianhong Li¹ Qinyi Sun¹ Lijie Fan² Kaiming He¹

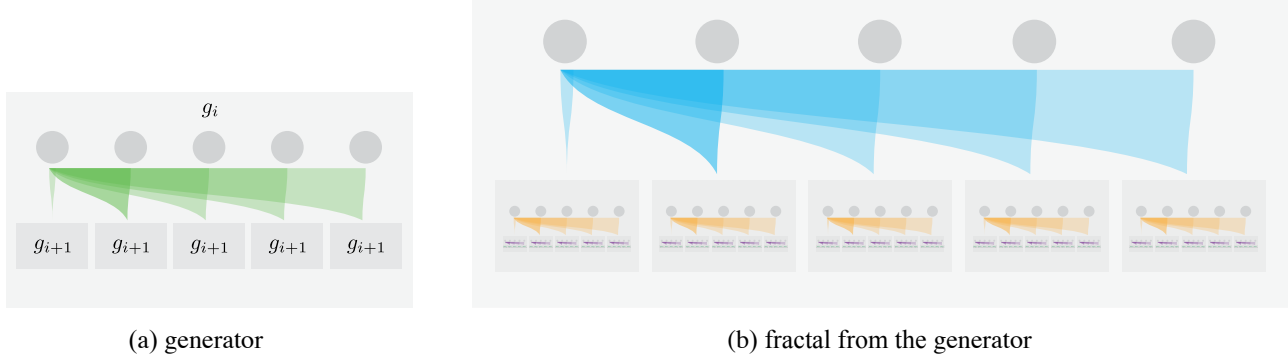


Figure 1: **Fractal Generative Model**. Four levels shown in this figure, better viewed zoomed in. In this instantiation, we use autoregressive model for the fractal generator. By recursively calling autoregressive models in autoregressive models, we build a fractal-like framework with self-similarity across different levels.

Abstract

Modularization is a cornerstone of computer science, abstracting complex functions into atomic building blocks. In this paper, we introduce a new level of modularization by abstracting generative models into atomic generative modules. Analogous to fractals in mathematics, our method constructs a new type of generative model by recursively invoking atomic generative modules, resulting in self-similar fractal architectures that we call *fractal generative models*. As a running example, we instantiate our fractal framework using autoregressive models as the atomic generative modules and examine it on the challenging task of pixel-by-pixel image generation, demonstrating strong performance in both likelihood estimation and generation quality. We hope this work could open a new paradigm in generative modeling and provide a fertile ground for future research. Code is available at <https://github.com/LTH14/fractalgen>.

1. Introduction

At the core of computer science lies the concept of modularization. For example, deep neural networks are built from atomic “layers” that serve as modular units (Szegedy et al., 2015). Similarly, modern generative models—such as diffu-

sion models (Song et al., 2020) and autoregressive models (Radford et al., 2018)—are built from atomic “generative steps”, each implemented by a deep neural network. By abstracting complex functions into these atomic building blocks, modularization allows us to create more intricate systems by composing these modules.

Building on this concept, we propose abstracting a generative model itself as a module to develop more advanced generative models. Specifically, we introduce a generative model constructed by recursively invoking generative models of the same kind within itself. This recursive strategy results in a generative framework that exhibits complex architectures with self-similarity across different levels of modules, as shown in Figure 1.

Our proposal is analogous to the concept of *fractals* (Mandelbrot, 1983) in mathematics. Fractals are self-similar patterns constructed using a recursive rule called a *generator*¹. Similarly, our framework is also built by a recursive process of calling generative models in generative models, exhibiting self-similarity across different levels. Therefore, we name our framework as “*fractal generative model*”.

Fractals or near-fractals are commonly observed patterns in biological neural networks. Multiple studies have provided evidence for fractal or scale-invariant small-world network organizations in the brain and its functional networks (Bassett et al., 2006; Sporns, 2006; Bullmore & Sporns, 2009).

¹MIT CSAIL ²Google DeemMind. Correspondence to: Tianhong Li <tianhong@mit.edu>, Kaiming He <kaiming@mit.edu>.

¹We use the term “generator” following (Mandelbrot, 1983). In this paper, “generator” specifically refers to a rule for recursively forming fractals.

These findings suggest that the brain’s development largely adopts the concept of modularization, recursively building larger neural networks from smaller ones.

Beyond biological neural networks, natural data often exhibit fractal or near-fractal patterns. Common fractal patterns range from macroscopic structures such as clouds, tree branches, and snowflakes, to microscopic ones including crystals (Cannon et al., 2000), chromatin (Mirny, 2011), and proteins (Enright & Leitner, 2005). More generally, natural images can also be analogized to fractals. For example, an image is composed of sub-images that are themselves images (although they may follow different distributions). Accordingly, an image generative model can be composed of modules that are themselves image generative models.

The proposed fractal generative model is inspired by the fractal properties observed in both biological neural networks and natural data. Analogous to natural fractal structures, the key component of our design is the *generator* which defines the recursive generation rule. For example, such a generator can be an autoregressive model, as illustrated in Figure 1. In this instantiation, each autoregressive model is composed of modules that are themselves autoregressive models. Specifically, each parent autoregressive block spawns multiple child autoregressive blocks, and each child block further spawns more autoregressive blocks. The resulting architecture exhibits fractal-like, self-similar patterns across different levels.

We examine this fractal instantiation on a challenging testbed: pixel-by-pixel image generation. Existing methods that directly model the pixel sequence do not achieve satisfactory results in both likelihood estimation and generation quality (Hawthorne et al., 2022; Yu et al., 2023), as images do not embody a clear sequential order. Despite its difficulty, pixel-by-pixel generation represents a broader class of important generative problems: modeling non-sequential data with intrinsic structures, which is particularly important for many data types beyond images, such as molecular structures, proteins, and biological neural networks.

Our proposed fractal framework demonstrates strong performance on this challenging yet important task. It can generate raw images pixel-by-pixel (Figure 2) while achieving accurate likelihood estimation and high generation quality. We hope our promising results will encourage further research on both the design and applications of fractal generative models, ultimately establishing a new paradigm in generative modeling.

2. Related Work

Fractals. A *fractal* is a geometric structure characterized by self-similarity across different scales, often constructed by a recursive generation rule called a generator (Mandelbrot,

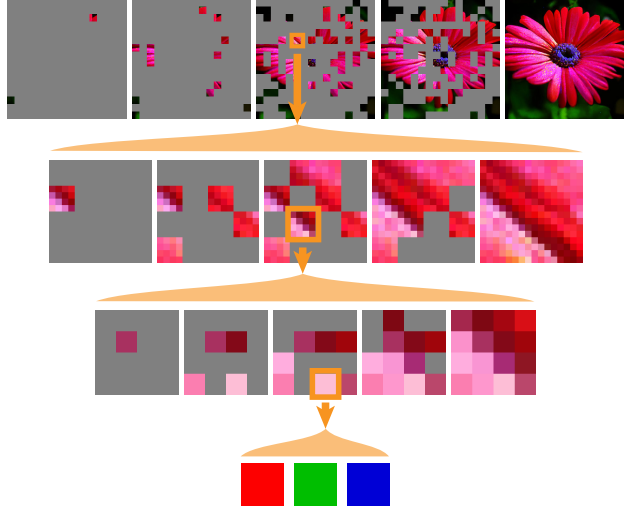


Figure 2: **Our fractal framework can generate high-quality images pixel-by-pixel.** We show the generation process of a 256×256 image by recursively calling autoregressive models in autoregressive models. We also provide example videos in our GitHub repository to illustrate the generation process.

1983). Fractals are widely observed in nature, with classic examples ranging from macroscopic structures such as clouds, tree branches, and snowflakes, to microscopic ones including crystals (Cannon et al., 2000), chromatin (Mirny, 2011), and proteins (Enright & Leitner, 2005).

Beyond these more easily recognizable fractals, many natural data also exhibit near-fractal characteristics. Although they do not possess strict self-similarity, they still embody similar multi-scale representations or patterns, such as images (Freeman et al., 1991; Lowe, 1999) and biological neural networks (Bassett et al., 2006; Sporns, 2006; Bullmore & Sporns, 2009). Conceptually, our fractal generative model naturally accommodates all such non-sequential data with intrinsic structures and self-similarity across different scales; in this paper, we demonstrate its capabilities with an image-based instantiation.

Due to its recursive generative rule, fractals inherently exhibit hierarchical structures that are conceptually related to hierarchical design principles in computer vision. However, most hierarchical methods in computer vision do not incorporate the recursion or divide-and-conquer paradigm fundamental to fractal construction, nor do they display self-similarity in their design. The unique combination of hierarchical structure, self-similarity, and recursion is what distinguishes our fractal framework from the hierarchical methods discussed next.

Hierarchical Representations. Extracting hierarchical pyramid representations from visual data has long been

an important topic in computer vision. Many early hand-engineered features, such as Steerable Filters, Laplacian Pyramid, and SIFT, employ scale-space analysis to construct feature pyramids (Burt & Adelson, 1987; Freeman et al., 1991; Lowe, 1999; 2004; Dalal & Triggs, 2005). In the context of neural networks, hierarchical designs remain important for capturing multi-scale information. For instance, SPPNet (He et al., 2015) and FPN (Lin et al., 2017) construct multi-scale feature hierarchies with pyramidal feature maps. Our fractal framework is also related to Swin Transformer (Liu et al., 2021), which builds hierarchical feature maps by attending to local windows at different scales. These hierarchical representations have proven effective in various image understanding tasks, including image classification, object detection, and semantic segmentation.

Hierarchical Generative Models. Hierarchical designs are also widely used in generative modeling. Many recent methods employ a two-stage paradigm, where a pre-trained tokenizer maps images into a compact latent space, followed by a generative model on those latent codes (van den Oord et al., 2017; Razavi et al., 2019; Esser et al., 2021; Ramesh et al., 2021). Another example, MegaByte (Yu et al., 2023), implements a two-scale model with a global and a local module for more efficient autoregressive modeling of long pixel sequences, though its performance remains limited.

Another line of research focuses on scale-space image generation. Cascaded diffusion models (Ramesh et al., 2022; Saharia et al., 2022; Pernias et al., 2023) train multiple diffusion models to progressively generate images from low resolution to high resolution. More recently, scale-space autoregressive methods (Tian et al., 2024; Tang et al., 2024; Han et al., 2024) generate tokens one scale at a time using an autoregressive transformer. However, generating images without a tokenizer is often prohibitively expensive for these autoregressive approaches because the large number of tokens or pixels per scale leads to a quadratic computational cost for the attention within each scale.

Modularized Neural Architecture Design. Modularization is a fundamental concept in computer science and deep learning, which atomizes previously complex functions into simple modular units. One of the earliest modular neural architectures was GoogleNet (Szegedy et al., 2015), which introduced the “Inception module” as a new level of organization. Later research expanded on this principle, designing widely used units such as the residual block (He et al., 2016) and the Transformer block (Vaswani, 2017). Recently, in the field of generative modeling, MAR (Li et al., 2024) modularize diffusion models as atomic building blocks to model the distribution of each continuous token, enabling the autoregressive modeling of continuous data. By providing higher levels of abstraction, modularization enables us to build more intricate and advanced neural architectures using

existing methods as building blocks.

A pioneering approach that applies a modular unit recursively and integrates fractal concepts in neural architecture design is FractalNet (Larsson et al., 2016), which constructs very deep neural networks by recursively calling a simple expansion rule. While FractalNet shares our core idea of recursively invoking a modular unit to form fractal structures, it differs from our method in two key aspects. First, FractalNet uses a small block of convolutional layers as its modular unit, while we use an entire generative model, representing different levels of modularization. Second, FractalNet was mainly designed for classification tasks and thus outputs only low-dimensional logits. In contrast, our approach leverages the exponential scaling behavior of fractal patterns to generate a large set of outputs (e.g., millions of image pixels), demonstrating the potential of fractal-inspired designs for more complex tasks beyond classification.

3. Fractal Generative Models

The key idea behind fractal generative models is to recursively construct more advanced generative models from existing atomic generative modules. In this section, we first present the high-level motivation and intuition behind fractal generative models. We then use the autoregressive model as an illustrative atomic module to demonstrate how fractal generative models can be instantiated and used to model very high-dimensional data distributions.

3.1. Motivation and Intuition

Fractals are complex patterns that emerge from simple, recursive rules. In fractal geometry, these rules are often called “generators” (Mandelbrot, 1983). With different generators, fractal methods can construct many natural patterns, such as clouds, mountains, snowflakes, and tree branches, and have been linked to even more intricate systems, such as the structure of biological neural networks (Bassett et al., 2006; Sporns, 2006; Bullmore & Sporns, 2009), nonlinear dynamics (Aguirre et al., 2009), and chaotic systems (Mandelbrot et al., 2004).

Formally, a fractal generator g_i specifies how to produce a set of new data $\{x_{i+1}\}$ for the next-level generator based on one output x_i from the previous-level generator: $\{x_{i+1}\} = g_i(x_i)$. For example, as shown in Figure 1, a generator can construct a fractal by recursively calling similar generators within each grey box.

Because each generator level can produce multiple outputs from a single input, a fractal framework can achieve an exponential increase in generated outputs while only requiring a linear number of recursive levels. This property makes it particularly suitable for modeling high-dimensional data with relatively few generator levels.

Specifically, we introduce a fractal generative model that uses atomic generative modules as *parametric* fractal generators. In this way, a neural network “learns” the recursive rule directly from data. By combining the exponential growth of fractal outputs with neural generative modules, our fractal framework enables the modeling of high-dimensional non-sequential data. Next, we demonstrate how we instantiate this idea with an autoregressive model as the fractal generator.

3.2. Autoregressive Model as Fractal Generator

In this section, we illustrate how to use autoregressive models as the fractal generator to build a fractal generative model. Our goal is to model the joint distribution of a large set of random variables $x_1, \dots, x_N$, but directly modeling it with a single autoregressive model is computationally prohibitive. To address this, we adopt a divide-and-conquer strategy. The key modularization is to abstract an autoregressive model as a modular unit that models a probability distribution $p(x|c)$. With this modularization, we can construct a more powerful autoregressive model by building it on top of multiple next-level autoregressive models.

Assume that the sequence length in each autoregressive model is a manageable constant k , and let the total number of random variables $N = k^n$, where $n = \log_k(N)$ represents the number of recursive levels in our fractal framework. The first autoregressive level of the fractal framework then partitions the joint distribution into k subsets, each containing k^{n-1} variables. Formally, we decompose $p(x_1, \dots, x_{k^n}) = \prod_{i=1}^k p(x_{(i-1) \cdot k^{n-1} + 1}, \dots, x_{i \cdot k^{n-1}} | x_1, \dots, x_{(i-1) \cdot k^{n-1}})$. Each conditional distribution $p(\dots | \dots)$ with k^{n-1} variables is then modeled by the autoregressive model at the second recursive level, so on and so forth. By recursively calling such a divide-and-conquer process, our fractal framework can efficiently handle the joint distribution of k^n variables using n levels of autoregressive models, each operating on a manageable sequence length k .

This recursive process represents a standard divide-and-conquer strategy. By recursively decomposing the joint distribution, our fractal autoregressive architecture not only significantly reduces computational costs compared to a single large autoregressive model but also captures the intrinsic hierarchical structure within the data.

Conceptually, as long as the data exhibits a structure that can be organized in a divide-and-conquer manner, it can be naturally modeled within our fractal framework. To provide a more concrete example, in the next section, we implement this approach to tackle the challenging task of pixel-by-pixel image generation.

4. An Image Generation Instantiation

We now present one concrete implementation of fractal generative models with an instantiation on the challenging task of pixel-by-pixel image generation. Although we use image generation as a testbed in this paper, the same divide-and-conquer architecture could be potentially adapted to other data domains. Next, we first discuss the challenges and importance of pixel-by-pixel image generation.

4.1. Pixel-by-pixel Image Generation

Pixel-by-pixel image generation remains a significant challenge in generative modeling because of the high dimensionality and complexity of raw image data. This task requires models that can efficiently handle a large number of pixels while effectively learning the rich structural patterns and interdependency between them. As a result, pixel-by-pixel image generation has become a challenging benchmark where most existing methods are still limited to likelihood estimation and fail to generate satisfactory images (Child et al., 2019; Hawthorne et al., 2022; Yu et al., 2023).

Though challenging, pixel-by-pixel generation represents a broader class of important high-dimensional generative problems. These problems aim to generate data element-by-element but differ from long-sequence modeling in that they typically involve non-sequential data. For example, many structures—such as molecular configurations, proteins, and biological neural networks—do not exhibit a sequential architecture yet embody very high-dimensional and structural data distributions. By selecting pixel-by-pixel image generation as an instantiation for our fractal framework, we aim not only to tackle a pivotal challenge in computer vision but also to demonstrate the potential of our fractal approach in addressing the broader problem of modeling high-dimensional non-sequential data with intrinsic structures.

4.2. Architecture

As shown in Figure 3, each autoregressive model takes the output from the previous-level generator as its input and produces multiple outputs for the next-level generator. It also takes an image (which can be a patch of the original image), splits it into patches, and embeds them to form the input sequence for a transformer model. These patches are also fed to the corresponding next-level generators. The transformer then takes the output of the previous generator as a separate token, placed before the image tokens. Based on this combined sequence, the transformer produces multiple outputs for the next-level generator.

Following common practices from vision transformers and image generative models (Dosovitskiy et al., 2020; Peebles & Xie, 2023), we set the sequence length of the first generator g_0 to 256, dividing the original images into 16×16

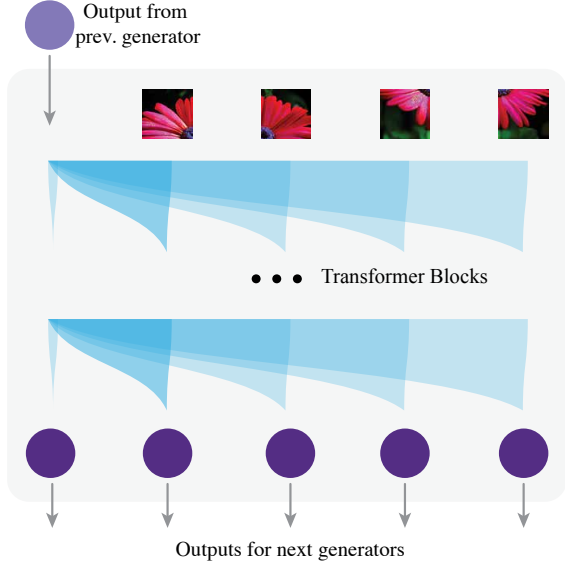


Figure 3: **Instantiation of our fractal method on pixel-by-pixel image generation.** In each fractal level, an autoregressive model receives the output from the previous generator, concatenates it with the corresponding image patches, and employs multiple transformer blocks to produce a set of outputs for the next generators.

patches. The second-level generator then models each patch and further subdivides them into smaller patches, continuing this process recursively. To manage computational costs, we progressively reduce the width and the number of transformer blocks for smaller patches, as modeling smaller patches is generally easier than larger ones. At the final level, we use a very lightweight transformer to model the RGB channels of each pixel autoregressively, and apply a 256-way cross-entropy loss on the prediction. The exact configurations and computational costs for each transformer across different recursive levels and resolutions are detailed in Table 1. Notably, with our fractal design, the computational cost of modeling a 256×256 image is only twice that of modeling a 64×64 image.

Following (Li et al., 2024), our method supports different autoregressive designs. In this work, we mainly consider two variants: a raster-order, GPT-like causal transformer (AR) and a random-order, BERT-like bidirectional transformer (MAR) (Figure 6). Both designs follow the autoregressive principle of next-token prediction, each with its own advantages and disadvantages, which we discuss in detail in Appendix B. We name the fractal framework using the AR variant as FractalAR and the MAR variant as FractalMAR.

4.3. Relation to Scale-space Autoregressive Models

Recently, several models have been introduced that perform next-scale prediction for autoregressive image generation

Table 1: **Model configurations, parameters, and computational costs** at different autoregressive levels for our large-size model. The computational costs are measured by the GFLOPs per forward pass in training with batch size 1. Notably, thanks to the fractal design, the total computational cost of modeling a 256×256 image is only twice that of modeling a 64×64 image.

		image resolution	
		$64 \times 64 \times 3$	$256 \times 256 \times 3$
seq. len.	g_1	256	256
	g_2	16	16
	g_3	3	16
	g_4	-	3
#layers	g_1	32	32
	g_2	8	8
	g_3	3	4
	g_4	-	1
hidden dim	g_1	1024	1024
	g_2	512	512
	g_3	128	256
	g_4	-	64
#params (M)	g_1	403	403
	g_2	25	25
	g_3	0.6	3
	g_4	-	0.1
#GFLOPs	g_1	215	215
	g_2	208	208
	g_3	15	419
	g_4	-	22

(Tian et al., 2024; Tang et al., 2024; Han et al., 2024). One major difference between these scale-space autoregressive models and our proposed method is that they use a single autoregressive model to predict tokens scale-by-scale. In contrast, our fractal framework adopts a divide-and-conquer strategy to recursively model raw pixels with generative submodules. Another key difference lies in the computational complexities: scale-space autoregressive models are required to perform *full attention* over the entire sequence of next-scale tokens when generating them, which results in substantially higher computational complexity.

For example, when generating images at a resolution of 256×256 , in the last scale, the attention matrix in each attention block of a scale-space autoregressive model has a size of $(256 \times 256)^2 = 4,294,967,296$. In contrast, our method performs attention on very small patches when modeling the interdependence of pixels (4×4), where each patch’s attention matrix is only $(4 \times 4)^2 = 256$, resulting in a total attention matrix size of $(64 \times 64) \times (4 \times 4)^2 = 1,048,576$ operations. This reduction makes our approach 4000 $\times$ more computationally efficient at the finest resolution and thus for the first time enables modeling high-resolution images pixel-by-pixel.

4.4. Relation to Long-Sequence Modeling

Most previous work on pixel-by-pixel generation formulates the problem as long-sequence modeling and leverages methods from language modeling to address it (Child et al., 2019; Roy et al., 2021; Ren et al., 2021; Hawthorne et al., 2022; Yu et al., 2023). However, the intrinsic structures of many data types, including but not limited to images, are beyond one-dimensional sequences. Different from these methods, we treat such data as sets (instead of sequences) composed of multiple elements and employ a divide-and-conquer strategy to recursively model smaller subsets with fewer elements. This approach is motivated by the observation that much of this data exhibits a near-fractal structure: images are composed of sub-images, molecules are composed of sub-molecules, and biological neural networks are composed of sub-networks. Accordingly, generative models designed to handle such data should be composed of submodules that are themselves generative models.

4.5. Implementation

We briefly describe the training and generation process of our fractal image generation framework. Further details and hyper-parameters can be found in [Appendix A](#).

Training. We train the fractal model end-to-end on raw image pixels by going through the fractal architecture in a breadth-first manner. During training, each autoregressive model takes in an input from the previous autoregressive model and produces a set of outputs for the next-level autoregressive model to take as inputs. This process continues down to the final level, where the image is represented as a sequence of pixels. A final autoregressive model uses the output on each pixel to predict the RGB channels in an autoregressive manner. We compute a cross-entropy loss on the predicted logits (treating the RGB values as discrete integers from 0 to 255) and backpropagate this loss through all levels of autoregressive models, thus training the entire fractal framework end-to-end.

Generation. Our fractal model generates the image in a pixel-by-pixel manner, following a depth-first order to go through the fractal architecture, as illustrated in [Figure 2](#). Here we use the random-order generation scheme from MAR (Li et al., 2024) as an example. The first-level autoregressive model captures the interdependence between 16×16 image patches, and at each step, it generates outputs for the next level based on known patches. The second-level model then leverages these outputs to model the interdependence between 4×4 patches within each 16×16 patch. Similarly, the third-level autoregressive model models the interdependence between individual pixels within each 4×4 patch. Finally, the last-level autoregressive model samples the actual RGB values from the autoregressively predicted RGB logits.

Table 2: **More fractal levels achieve better likelihood estimation performance with lower computational costs**, measured on unconditional ImageNet 64×64 test set. NLL is reported in bits/dim. The network configurations of g_1 , g_2 and g_3 are in [Table 1](#).

	Seq Len			#GFLOPs	NLL↓
	g_1	g_2	g_3		
AR, full-length	12288	-	-	29845	N/A
MAR, full-length	12288	-	-	29845	N/A
AR, 2-level	4096	3	-	5516	3.34
MAR, 2-level	4096	3	-	5516	3.36
FractalAR (3-level)	256	16	3	438	3.14
FractalMAR (3-level)	256	16	3	438	3.15

Table 3: Comparison with other likelihood-based models on unconditional ImageNet 64×64 test set. NLL is reported in bits/dim. Both FractalAR and FractalMAR achieve competitive performance. † We regard flow matching as a generalized form of diffusion model.

	type	NLL↓
iDDPM (Nichol & Dhariwal, 2021)	diffusion	3.53
VDM (Kingma et al., 2021)	diffusion	3.40
FM (Lipman et al., 2022)	diffusion†	3.31
NFDM (Bartosh et al., 2024)	diffusion	3.20
PixelRNN (van den Oord et al., 2016b)	AR	3.63
PixelCNN (van den Oord et al., 2016a)	AR	3.57
Sparse Transformer (Child et al., 2019)	AR	3.44
Routing Transformer (Roy et al., 2021)	AR	3.43
Combiner AR (Ren et al., 2021)	AR	3.42
Perceiver AR (Hawthorne et al., 2022)	AR	3.40
MegaByte (Yu et al., 2023)	AR	3.40
FractalAR	fractal	3.14
FractalMAR	fractal	3.15

5. Experiments

We conduct extensive experiments on the ImageNet dataset (Deng et al., 2009) with resolutions at 64×64 and 256×256 . Our evaluation includes both unconditional and class-conditional image generation, covering various aspects of the model such as likelihood estimation, fidelity, diversity, and generation quality. Accordingly, we report the negative log-likelihood (NLL), Frechet Inception Distance (FID) (Heusel et al., 2017), Inception Score (IS) (Salimans et al., 2016), Precision and Recall (Dhariwal & Nichol, 2021), and visualization results for a comprehensive assessment of our fractal framework.

5.1. Likelihood Estimation

We begin by evaluating our method on unconditional ImageNet 64×64 generation to assess its likelihood estimation capability. To examine the effectiveness of our fractal frame-



Figure 4: **Pixel-by-pixel generation results from FractalMAR-H on ImageNet 256×256 .** Our fractal method can generate high-quality high-resolution images in a pixel-by-pixel manner, with an average throughput of 1.29 seconds per image. More qualitative results are in [Figure 7](#).

work, we compare the likelihood estimation performance of our framework with different numbers of fractal levels, as shown in [Table 2](#). Modeling the entire $64 \times 64 \times 3 = 12,288$ -pixel sequence with a single autoregressive model incurs prohibitive computational costs and makes training infeasible. Moreover, a two-level fractal framework that first models the entire pixel sequence and then the RGB channels requires over ten times the computation of our three-level fractal model. Employing more fractal levels is not only more computationally efficient but also improves likelihood estimation performance, likely because it better captures the intrinsic hierarchical structure of images. These results demonstrate both the efficiency and effectiveness of our fractal framework.

We further compare our method with other likelihood-based models in [Table 5](#). Our fractal generative model, instantiated with both causal and masked autoregressive fractal generators, achieves strong likelihood performance. In particular, it achieves a negative log-likelihood of 3.14 bits per dim, which outperforms the previous best autoregressive model (3.40 bits per dim) by a significant margin and re-

mains competitive with advanced diffusion-based methods. These findings demonstrate the effectiveness of our fractal framework on the challenging task of pixel-by-pixel image generation, highlighting its promising potential in modeling high-dimensional non-sequential data distributions.

5.2. Generation Quality

We also evaluate FractalMAR on the challenging task of class-conditional image generation at a resolution of 256×256 , using four fractal levels. We report standard metrics including FID, Inception Score, Precision, and Recall following standard practices to evaluate its generation quality in [Table 4](#). Specifically, FractalMAR-H achieves an FID of 6.15 and an Inception Score of 348.9, with an average throughput of 1.29 seconds per image (evaluated at a batch size of 1,024 on a single Nvidia H100 PCIe GPU). Notably, our method achieves strong Inception Score and Precision, indicating its ability to generate images with high fidelity and fine-grained details, as also demonstrated in [Figure 4](#). However, its FID and Recall are relatively weaker, suggesting that the generated samples are less diverse com-

Table 4: Sample quality evaluation on ImageNet 256×256 generation. FractalMAR is the only **pixel-by-pixel** framework in this table. We also provide the performance of other pixel-level generation methods for reference.

	type	#params	FID↓	IS↑	Pre.↑	Rec.↑
BigGAN-deep	GAN	160M	6.95	198.2	0.87	0.28
GigaGAN	GAN	569M	3.45	225.5	0.84	0.61
StyleGAN-XL	GAN	166M	2.30	265.1	0.78	0.53
ADM	diffusion	554M	4.59	186.7	0.82	0.52
Simple diffusion	diffusion	2B	3.54	205.3	-	-
VDM++	diffusion	2B	2.12	267.7	-	-
SiD2	diffusion	-	1.38	-	-	-
JetFormer	AR+flow	2.8B	6.64	-	0.69	0.56
FractalMAR-B	fractal	186M	11.80	274.3	0.78	0.29
FractalMAR-L	fractal	438M	7.30	334.9	0.79	0.44
FractalMAR-H	fractal	848M	6.15	348.9	0.81	0.46

pared to other methods. We hypothesize that this is due to the significant challenge of modeling nearly 200,000 pixels in a pixel-by-pixel way. Nonetheless, these results highlight our method’s effectiveness in not only accurate likelihood estimation but also the generation of high-quality images.

We further observe a promising scaling trend: increasing the model size from 186M to 848M parameters substantially improves the FID from 11.80 to 6.15 and the Recall from 0.29 to 0.46. We expect that additional scaling of parameters could narrow the gap in FID and Recall even further. Unlike models that rely on tokenizers, our method is free from the reconstruction errors introduced by tokenization, suggesting the potential for uncapped performance gains with larger model capacities.

5.3. Conditional Pixel-by-pixel Prediction

We further examine the conditional pixel-by-pixel prediction performance of our method using conventional tasks in image editing. Figure 5 provides several examples, including inpainting, outpainting, uncropping, and class-conditional editing. As shown in the figure, our method can accurately predict the masked pixels based on the unmasked regions. Additionally, it can effectively capture high-level semantics from class labels and reflect it in the predicted pixels. This is illustrated in the class-conditional editing example, where the model replaces a cat’s face with that of a dog by conditioning on the dog’s class label. These results demonstrate the effectiveness of our method in predicting unknown data given known conditions.

More broadly, by generating data element-by-element, our method provides a generation process that is more interpretable for humans compared to approaches such as diffusion models or generative models operating in latent spaces. This interpretable generation process not only allows us to better understand how data is generated but also offers a way

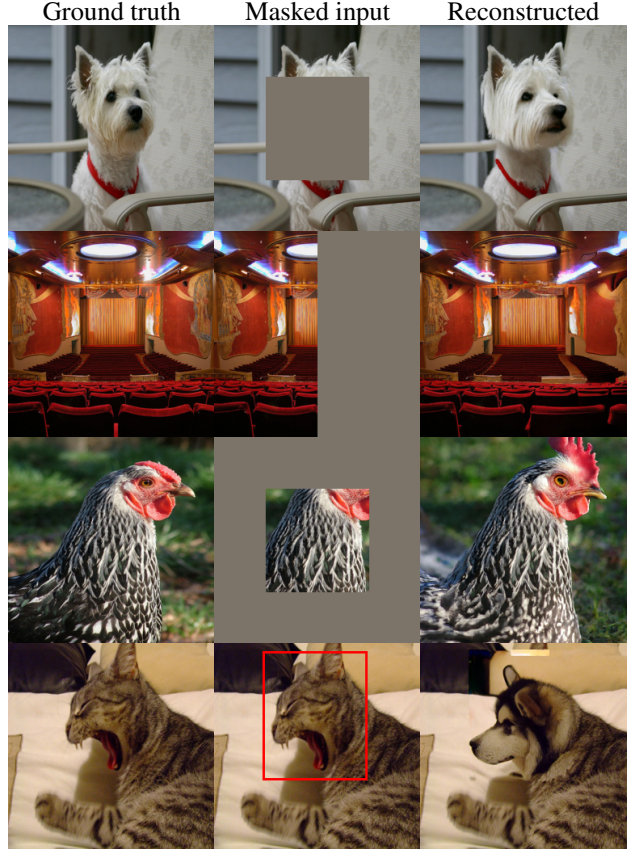


Figure 5: Conditional pixel-by-pixel prediction results, including image inpainting (first row), outpainting (second row), uncropping (outpainting on a large mask, third row), and class-conditional editing (inpainting with another class label, fourth row).

to control and interact with the generation. Such capabilities are particularly important in applications like visual content creation, architectural design, and drug discovery. Our promising results highlight the potential of our approach for controllable and interactive generation, paving the way for future explorations in this direction.

6. Discussion and Conclusion

The effectiveness of our proposed fractal generative model, demonstrated through the challenging task of pixel-by-pixel generation, suggests new opportunities for designing generative models. It highlights the potential of dividing complex data distributions into manageable sub-problems and addressing them by abstracting existing generative models into modular units. We believe that fractal generative models are particularly well-suited for modeling data with intrinsic structures beyond one-dimensional sequential orders. We hope that the simplicity and effectiveness of our method will inspire the research community to explore novel designs and applications of fractal generative models.

Broader Impacts. Our primary aim is to advance the fundamental research on generative models. Similar to all generative models, there are potential negative societal consequences if our approach is misused to produce disinformation or amplify biases. However, these considerations lie outside the scope of this paper and are therefore not discussed in detail.

Acknowledgements. We thank Google TPU Research Cloud (TRC) for granting us access to TPUs, and Google Cloud Platform for supporting GPU resources.

References

- Aguirre, J., Viana, R. L., and Sanjuán, M. A. Fractal structures in nonlinear dynamics. *Reviews of Modern Physics*, 81(1): 333–386, 2009.
- Bartosh, G., Vetrov, D., and Naesseth, C. A. Neural flow diffusion models: Learnable forward process for improved diffusion modelling. *arXiv preprint arXiv:2404.12940*, 2024.
- Bassett, D. S., Meyer-Lindenberg, A., Achard, S., Duke, T., and Bullmore, E. Adaptive reconfiguration of fractal small-world human brain functional networks. *Proceedings of the National Academy of Sciences*, 103(51):19518–19523, 2006.
- Bullmore, E. and Sporns, O. Complex brain networks: graph theoretical analysis of structural and functional systems. *Nature reviews neuroscience*, 10(3):186–198, 2009.
- Burt, P. J. and Adelson, E. H. The laplacian pyramid as a compact image code. In *Readings in computer vision*, pp. 671–679. Elsevier, 1987.
- Cannon, J. W., Floyd, W. J., and Parry, W. R. Crystal growth, biological cell growth, and geometry. *Pattern formation in biology: vision, and dynamics*. Singapore: World Scientific, pp. 65–82, 2000.
- Chang, H., Zhang, H., Jiang, L., Liu, C., and Freeman, W. T. MaskGIT: Masked generative image Transformer. In *CVPR*, 2022.
- Chang, H., Zhang, H., Barber, J., Maschinot, A., Lezama, J., Jiang, L., Yang, M.-H., Murphy, K., Freeman, W. T., Rubinstein, M., Li, Y., and Krishnan, D. Muse: Text-to-image generation via masked generative Transformers. In *ICML*, 2023.
- Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- Dalal, N. and Triggs, B. Histograms of oriented gradients for human detection. In *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR’05)*, volume 1, pp. 886–893. Ieee, 2005.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009.
- Dhariwal, P. and Nichol, A. Diffusion models beat GANs on image synthesis. In *NeurIPS*, 2021.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- Enright, M. B. and Leitner, D. M. Mass fractal dimension and the compactness of proteins. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, 71(1):011912, 2005.
- Esser, P., Rombach, R., and Ommer, B. Taming Transformers for high-resolution image synthesis. In *CVPR*, 2021.
- Freeman, W. T., Adelson, E. H., et al. The design and use of steerable filters. *IEEE Transactions on Pattern analysis and machine intelligence*, 13(9):891–906, 1991.
- Goyal, P., Dollár, P., Girshick, R., Noordhuis, P., Wesolowski, L., Kyrola, A., Tulloch, A., Jia, Y., and He, K. Accurate, large mini-batch SGD: Training ImageNet in 1 hour. *arXiv:1706.02677*, 2017.
- Han, J., Liu, J., Jiang, Y., Yan, B., Zhang, Y., Yuan, Z., Peng, B., and Liu, X. Infinity: Scaling bitwise autoregressive modeling for high-resolution image synthesis. *arXiv preprint arXiv:2412.04431*, 2024.
- Hawthorne, C., Jaegle, A., Cangea, C., Borgeaud, S., Nash, C., Malinowski, M., Dieleman, S., Vinyals, O., Botvinick, M., Simon, I., et al. General-purpose, long-context autoregressive modeling with perceiver ar. In *International Conference on Machine Learning*, pp. 8535–8558. PMLR, 2022.
- He, K., Zhang, X., Ren, S., and Sun, J. Spatial pyramid pooling in deep convolutional networks for visual recognition. *IEEE transactions on pattern analysis and machine intelligence*, 37(9):1904–1916, 2015.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *CVPR*, 2016.
- Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., and Hochreiter, S. GANs trained by a two time-scale update rule converge to a local nash equilibrium. In *NIP*, 2017.
- Ho, J. and Salimans, T. Classifier-free diffusion guidance. *arXiv:2207.12598*, 2022.
- Kingma, D., Salimans, T., Poole, B., and Ho, J. Variational diffusion models. volume 34, pp. 21696–21707, 2021.
- Larsson, G., Maire, M., and Shakhnarovich, G. Fractalnet: Ultra-deep neural networks without residuals. *arXiv preprint arXiv:1605.07648*, 2016.
- Li, T., Chang, H., Mishra, S., Zhang, H., Katabi, D., and Krishnan, D. MAGE: Masked generative encoder to unify representation learning and image synthesis. In *CVPR*, 2023.
- Li, T., Tian, Y., Li, H., Deng, M., and He, K. Autoregressive image generation without vector quantization. *arXiv preprint arXiv:2406.11838*, 2024.
- Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., and Belongie, S. Feature pyramid networks for object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2117–2125, 2017.

- Lipman, Y., Chen, R. T., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., and Guo, B. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 10012–10022, 2021.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *ICLR*, 2019.
- Lowe, D. G. Object recognition from local scale-invariant features. In *Proceedings of the seventh IEEE international conference on computer vision*, volume 2, pp. 1150–1157. Ieee, 1999.
- Lowe, D. G. Distinctive image features from scale-invariant keypoints. *International journal of computer vision*, 60:91–110, 2004.
- Mandelbrot, B. B. The fractal geometry of nature/revised and enlarged edition. *New York*, 1983.
- Mandelbrot, B. B., Evertsz, C. J., and Gutzwiller, M. C. *Fractals and chaos: the Mandelbrot set and beyond*, volume 3. Springer, 2004.
- Mirny, L. A. The fractal globule as a model of chromatin architecture in the cell. *Chromosome research*, 19:37–51, 2011.
- Mustafi, D., Engel, A. H., and Palczewski, K. Structure of cone photoreceptors. *Progress in retinal and eye research*, 28(4): 289–302, 2009.
- Nichol, A. Q. and Dhariwal, P. Improved denoising diffusion probabilistic models. In *ICML*, 2021.
- Peebles, W. and Xie, S. Scalable diffusion models with Transformers. In *ICCV*, 2023.
- Pernias, P., Rampas, D., Richter, M. L., Pal, C. J., and Aubreville, M. Würstchen: An efficient architecture for large-scale text-to-image diffusion models. *arXiv preprint arXiv:2306.00637*, 2023.
- Radford, A., Narasimhan, K., Salimans, T., and Sutskever, I. Improving language understanding by generative pre-training. 2018.
- Ramesh, A., Pavlov, M., Goh, G., Gray, S., Voss, C., Radford, A., Chen, M., and Sutskever, I. Zero-shot text-to-image generation. In *ICML*, 2021.
- Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., and Chen, M. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*, 2022.
- Razavi, A., Van den Oord, A., and Vinyals, O. Generating diverse high-fidelity images with VQ-VAE-2. In *NeurIPS*, 2019.
- Ren, H., Dai, H., Dai, Z., Yang, M., Leskovec, J., Schuurmans, D., and Dai, B. Combiner: Full attention transformer with sparse computation cost. *Advances in Neural Information Processing Systems*, 34:22470–22482, 2021.
- Roy, A., Saffar, M., Vaswani, A., and Grangier, D. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9: 53–68, 2021.
- Saharia, C., Chan, W., Saxena, S., Li, L., Whang, J., Denton, E. L., Ghasemipour, K., Gontijo Lopes, R., Karagol Ayan, B., Salimans, T., et al. Photorealistic text-to-image diffusion models with deep language understanding. *NeurIPS*, 2022.
- Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., and Chen, X. Improved techniques for training GANs. In *NeurIPS*, 2016.
- Sauer, A., Schwarz, K., and Geiger, A. Stylegan-xl: Scaling stylegan to large diverse datasets. In *ACM SIGGRAPH 2022 conference proceedings*, pp. 1–10, 2022.
- Song, J., Meng, C., and Ermon, S. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.
- Song, Y., Dhariwal, P., Chen, M., and Sutskever, I. Consistency models. *arXiv preprint arXiv:2303.01469*, 2023.
- Sporns, O. Small-world connectivity, motif composition, and complexity of fractal neuronal connections. *Biosystems*, 85(1): 55–64, 2006.
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., and Rabinovich, A. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1–9, 2015.
- Tang, H., Wu, Y., Yang, S., Xie, E., Chen, J., Chen, J., Zhang, Z., Cai, H., Lu, Y., and Han, S. Hart: Efficient visual generation with hybrid autoregressive transformer. *arXiv preprint arXiv:2410.10812*, 2024.
- Tian, K., Jiang, Y., Yuan, Z., Peng, B., and Wang, L. Visual autoregressive modeling: Scalable image generation via next-scale prediction. *arXiv preprint arXiv:2404.02905*, 2024.
- van den Oord, A., Kalchbrenner, N., Espeholt, L., Vinyals, O., Graves, A., and Kavukcuoglu, K. Conditional image generation with PixelCNN decoders. In *NeurIPS*, 2016a.
- van den Oord, A., Kalchbrenner, N., and Kavukcuoglu, K. Pixel recurrent neural networks. In *ICML*, 2016b.
- van den Oord, A., Vinyals, O., and Kavukcuoglu, K. Neural discrete representation learning. In *NeurIPS*, 2017.
- Vaswani, A. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- Yu, L., Simig, D., Flaherty, C., Aghajanyan, A., Zettlemoyer, L., and Lewis, M. Megabyte: Predicting million-byte sequences with multiscale transformers. *Advances in Neural Information Processing Systems*, 36:78808–78823, 2023.

A. Implementation Details

Here we provide more implementation details of the training and generation process of our fractal generative model.

Training. When modeling high-resolution images with multiple levels of autoregressive models, we find it slightly helpful to include a “guiding pixel” in the autoregressive sequence. Specifically, the model first uses the output from the previous generator to predict the average pixel value of the current input image. This average pixel value then serves as an additional condition for the transformer. In this way, each generator begins with a global context before predicting fine-grained details. We apply this “guiding pixel” only in experiments on ImageNet 256×256 .

Since our autoregressive model divides images into patches, it can lead to inconsistent patch boundaries during generation. To address this issue, we provide the next-level generator not only the output at the current patch but also the outputs at the surrounding four patches. Although incorporating these additional inputs slightly increases the sequence length, it significantly reduces the patch boundary artifacts.

By default, the models are trained using the AdamW optimizer (Loshchilov & Hutter, 2019) for 800 epochs (the FractalMAR-H model is trained for 600 epochs). The weight decay and momenta for AdamW are 0.05 and (0.9, 0.95). We use a batch size of 2048 for ImageNet 64×64 and 1024 for ImageNet 256×256 , and a base learning rate (lr) of $5e-5$ (scaled by batch size divided by 256). The model is trained with 40 epochs linear lr warmup (Goyal et al., 2017), followed by a cosine lr schedule.

Generation. Following common practices in the literature (Chang et al., 2022; 2023; Li et al., 2023), we implement classifier-free guidance (CFG) and temperature scaling for class-conditional generation. To facilitate CFG (Ho & Salimans, 2022), during training, the class condition is replaced with a dummy class token in 10% of the samples. At inference time, the model is run with both the given class token and the dummy token, yielding two sets of logits l_c and l_u for each pixel channel. The predicted logits are then adjusted according to the following equation: $l = l_u + \omega \cdot (l_c - l_u)$, where ω is the guidance scale. We employ a linear CFG schedule during inference for the first-level autoregressive model, as described in (Chang et al., 2023). We perform a sweep over different combinations of guidance scale and temperature to identify the optimal settings for each conditional generation model.

We also observed that CFG can suffer from numerical instability when the predicted probability for a pixel value is very small. To mitigate this issue, we apply top-p sampling with a threshold of 0.0001 on the conditional logits before applying CFG.

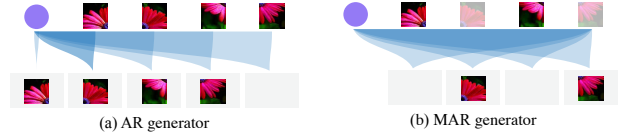


Figure 6: Two variants for autoregressive modeling. The AR variant models the sequence in a raster-scan order using a causal transformer, while the MAR variant models the sequence in a random order using a bidirectional transformer. Both are valid generators to build our fractal framework.

B. Additional Results

Table 5: Comparison on conditional ImageNet 64×64 with classical generative models.

	type	FID↓
iDDPM (Nichol & Dhariwal, 2021)	diffusion	2.92
StyleGAN-XL (Sauer et al., 2022)	GAN	1.51
Consistency Model (Song et al., 2023)	consistency	4.70
MAR (Li et al., 2024)	AR+diffusion	2.93
FractalAR	fractal	5.30
FractalMAR	fractal	2.72

Class-conditional ImageNet 64×64 . We evaluate class-conditional image generation on ImageNet 64×64 , reporting FID according to standard practice. Consistent with PixelCNN (van den Oord et al., 2016a), we find that class conditioning has negligible impact on NLL but substantially improves visual quality and FID. The results demonstrate that our fractal generative model can achieve competitive performance as classical generative models.

We also compare the performance of the AR and MAR variants, whose structures are illustrated in Figure 6. The AR variant leverages key-value caching to accelerate generation, whereas the MAR variant employs bidirectional attention, which aligns more naturally with image modeling and enables the parallel prediction of multiple patches, thereby improving computational efficiency. As shown in the table, both of our models achieve favorable performance, with FractalMAR outperforming FractalAR overall, as also demonstrated in (Li et al., 2024). Therefore, we choose to use the MAR variant for conditional image synthesis on ImageNet at a resolution of 256×256 .

Modeling Pixels. We also examine how different pixel modeling orders affect performance. We experimented with three autoregressive orders: RGB, GRB, and BGR, as well as converting the RGB channels to the YCbCr color space. The results are summarized in Table 6. We found that while all orders achieved similar negative log-likelihood values, the FID varied slightly among the autoregressive orders (note that the NLL in the YCbCr space is not comparable with that in the RGB space). This variation is likely be-

Table 6: Different orders of channel modeling within each pixel can slightly influence generation performance. All models are trained for 400 epochs on ImageNet 64×64 .

order	NLL↓	FID↓
Y→Cb→Cr	N/A	3.55
B→R→G	3.17	3.32
G→R→B	3.17	3.14
R→G→B	3.17	3.15

cause, akin to human vision, the Inception model used to compute FID places greater emphasis on the red and green channels than on the blue channels (Mustafi et al., 2009). Nonetheless, the choice of autoregressive order does not result in significant performance differences, demonstrating the robustness of our method.

Additional qualitative results. We further provide additional conditional generation results on ImageNet 256×256 in Figure 7, and additional conditional pixel-by-pixel prediction results in Figure 8.



Figure 7: Additional pixel-by-pixel generation results from FractalMAR-H on ImageNet 256×256 .

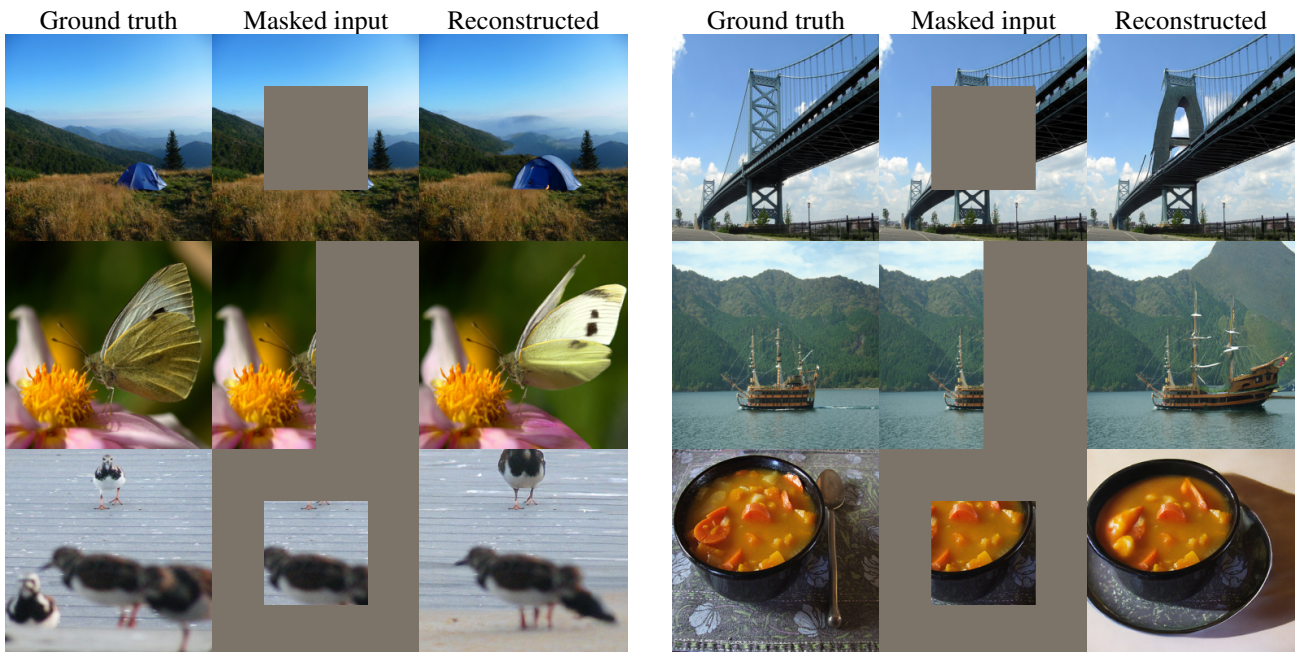


Figure 8: Additional conditional pixel-by-pixel prediction results, including image inpainting, outpainting, and uncropping.